

Artificial Intelligence Fundamentals

Project Report: Autonomous Exoplanetary Digital Twin (Hack-4-Sages)

Team / Organization: HardCounter
Project Repository: Hack - 4 - Sages
Date: [Fill in date]
Prepared by: [Fill in author(s)]

History of changes

Version	Date	Who	Description
01	[YYYY-MM-DD]	[Name]	Initial full draft based on current repository implementation
02	[YYYY-MM-DD]	[Name]	Added screenshots and simulation outputs
03	[YYYY-MM-DD]	[Name]	Final proofreading and formatting

Responsibilities

Name	Tasks
Aleksander	[Fill in exact responsibilities]
Denis	[Fill in exact responsibilities]
Piotr	[Fill in exact responsibilities]
Rafal	[Fill in exact responsibilities]

Table of Contents

1.

Description of a problem
2.

An analysis of a problem
3.

Existing solutions
4.

The description of the preferred solution
5.

Implementation of the AI part
6.

Simulation results from the application
7.

Conclusions
- Literature

1. Description of a problem

The rapid growth of exoplanet catalogs creates a practical scientific bottleneck: there are thousands of confirmed planets, but detailed climate interpretation for each one is expensive and slow. Full General Circulation Models (GCMs) are physically rich but computationally heavy, while very simple calculators are fast but too limited for nuanced habitability analysis.

The Hack-4-Sages project addresses this gap by building an interactive, AI-assisted exoplanet analysis platform that combines:

- real observational data retrieval (NASA Exoplanet Archive via TAP/ADQL),
- deterministic astrophysics calculations (temperature, habitability indices, habitable zone),
- fast surrogate climate simulation,
- uncertainty-aware and literature-aware interpretation,
- and robust fallback behavior when advanced modules are unavailable.

In practical terms, the user should be able to ask a natural-language question (for example, "Analyze TRAPPIST-1 e"), run a simulation in seconds, and obtain both numerical outputs and a scientifically grounded explanation.

Figure 1. [Insert screenshot: Main application view with all tabs visible.]

The core problem is therefore not only "compute one metric," but build a full decision-support workflow:

1. retrieve data,
2. validate inputs,
3. compute physically constrained indicators,
4. generate climate maps,
5. communicate results clearly for both expert and outreach audiences.

2. An analysis of a problem

From an engineering perspective, the problem splits into four major challenges.

2.1 Scientific validity under uncertainty

Habitability cannot be inferred from one number. The system must combine multiple signals (e.g., equilibrium temperature, ESI, SEPHI, habitable-zone placement, atmospheric retention heuristics, biosignature false-positive risk). Each metric has assumptions and limitations, so the application needs transparent outputs rather than "black-box scores."

2.2 Computational constraints

Fast response is necessary for interactive exploration. A user interface with sliders, catalog search, and live what-if analysis requires sub-second to few-second behavior for most operations. This excludes direct use of heavy 3D climate solvers as the default path.

2.3 Data quality and imbalance

Exoplanet datasets are incomplete and imbalanced. Potentially habitable examples are rare compared to the full population. This affects both model training and interpretation quality, and motivates synthetic data augmentation plus explicit post-generation physical filtering.

2.4 Reliability in real-world usage

The platform depends on optional components (trained models, Ollama, GPU availability, internet APIs). A production-grade demo cannot fail hard when one dependency is missing. Instead, it must degrade gracefully while preserving core deterministic functionality.

Figure 2. [Insert screenshot: Manual parameterization interface.]

Figure 3. [Insert screenshot: Simulation pipeline progress/status.]

This analysis led to a hybrid architecture: deterministic physics + lightweight ML surrogates + optional LLM orchestration, with strict validation and fallback logic at each stage.

3. Existing solutions

Several classes of existing solutions are relevant.

3.1 Classical climate modeling (GCM)

GCM approaches provide high physical fidelity and rich atmospheric dynamics, but they are too slow for interactive catalog-scale exploration. They are ideal for deep studies of selected targets, not rapid triage of many planets.

3.2 Single-metric calculators

Simple tools based on equilibrium temperature or basic habitability formulas are fast and easy to use, but they do not capture multi-factor interactions (e.g., day-night contrasts, interior-atmosphere coupling heuristics, photochemical false positives).

3.3 Catalog-only exploration tools

Archive interfaces (including ADQL-based data access) are strong for data retrieval but typically do not provide integrated physics simulation, AI interpretation, and interactive climate visualization in one place.

3.4 General-purpose LLM chat interfaces

Generic chat models can explain concepts but may hallucinate physical values if not grounded in deterministic tools and validated inputs.

The preferred direction is therefore a combined pipeline where the language model reads deterministic outputs instead of inventing them.

Figure 4. [Insert screenshot: Catalog search with generated ADQL and returned planets.]

4. The description of the preferred solution

The selected solution is the **Autonomous Exoplanetary Digital Twin**, implemented as a Streamlit application with five coordinated tabs:

- **Agent AI:** natural-language interaction with transparent reasoning chain,

- **Manual Mode:** slider-driven simulation and climate visualization,
- **Catalog:** NASA archive browsing, query translation, anomaly analysis,
- **Science:** advanced interpretation dashboard (HZ, ISA, false positives, uncertainty),
- **System:** runtime mode selection, diagnostics, export, architecture view.

4.1 Runtime profiles

The project supports three runtime modes:

- **Dual-LLM:** Qwen 2.5-14B orchestrator + AstroSage domain expert,
- **Single-LLM:** AstroSage handles both orchestration and interpretation,
- **Deterministic:** no LLM, physics/ML/visualization still available.

This design allows operation on different hardware profiles while preserving core scientific functionality.

4.2 High-level pipeline

1. Input or query enters the system.
2. Parameters are validated (Pydantic constraints).
3. Physics metrics are computed.
4. Climate map is generated (ELM / PINNFormer / analytical fallback).
5. Outputs are visualized (3D globe / 2D heatmap).
6. Optional LLM layers interpret results and provide citations.

4.3 Robustness strategy

A graceful degradation manager catches failures and switches to simpler alternatives (for example, trained model missing -> fallback model -> analytical generation) instead of crashing the app.

Figure 5. [Insert screenshot: Runtime mode selection in System tab.]

Figure 6. [Insert screenshot: Architecture diagram from System tab.]

5. Implementation of the AI part

The AI layer in this project is not a single model; it is a coordinated set of deterministic and learned components.

5.1 Physics computation engine

The module `modules/astro_physics.py` implements core formulas and indices, including:

- equilibrium temperature (T_{eq}),
- stellar flux (absolute and Earth-normalized),
- Earth Similarity Index (ESI),
- SEPHI criteria,
- habitable-zone boundaries (Kopparapu-type parameterization),
- habitable surface fraction from temperature maps,
- ISA interaction heuristics (outgassing, tectonics plausibility, volatile retention),
- biosignature false-positive risk heuristics (including UV environment context),

- composition-related helpers (radius-gap class, sulfur chemistry, C/O interpretation).

These deterministic calculations define the scientific backbone that other modules consume.

5.2 ELM climate surrogate

`modules/elm_surrogate.py` provides an ensemble-based Extreme Learning Machine implementation:

- feature preparation from planetary and stellar parameters,
- fast training via analytical output-layer solution (Moore-Penrose style),
- ensemble predictions for variance reduction,
- conformal-style prediction intervals (mean/lower/upper maps),
- serialization for deployment (`models/elm_ensemble.pkl`).

This model is the fast default for climate map generation.

5.3 PINNFormer 3-D

`modules/pinnformer3d.py` implements a transformer-based physics-informed neural approach with configurable physics modes (basic, greenhouse, OHT, clouds, tidal, ice-albedo, advection, full combinations).

When available, it provides richer physics-driven fields and can output additional layers such as cloud/ice/ocean diagnostics.

5.4 Data augmentation and anomaly analysis

- `modules/data_augmentation.py` trains CTGAN on normalized planetary features and validates synthetic samples with physical filters and percentile clipping.
- `modules/anomaly_detection.py` applies Isolation Forest and UMAP to identify unusual planets and visualize population structure.

5.5 Agent orchestration and RAG

`modules/agent_setup.py` defines an LLM agent with tool access. Tools include (among others):

- NASA query,
- habitability computation,
- climate simulation,
- radius-gap classification,
- sulfur chemistry prediction,
- C/O assessment,
- domain expert consultation,
- habitable-candidate ranking,
- two-planet comparison,
- anomaly detection,
- scientific citation retrieval.

`modules/rag_citations.py` contains a curated paper corpus and retrieval logic for citation-backed responses.

5.6 Validation and guardrails

`modules/validators.py` applies strict physical constraints for stellar parameters, planetary parameters, and simulation outputs. This prevents unphysical values from propagating through the pipeline.

Figure 7. *[Insert screenshot: Agent response with transparent reasoning chain.]*

Figure 8. *[Insert screenshot: Core simulation metrics (ESI, SEPHI, HSF, flux).]*

Figure 9. *[Insert screenshot: 3D climate globe visualization.]*

Figure 10. *[Insert screenshot: Science dashboard overview.]*

6. Simulation results from the application

This section should document reproducible scenarios. Suggested structure:

6.1 Scenario A: Earth-like sanity case

Use approximate Sun/Earth parameters to verify that output ranges are physically plausible (especially T_{eq}), ESI, and SEPHI flags).

Record:

- input parameters,
- resulting T_{eq}), ESI, SEPHI, HSF, flux,
- selected climate method (ELM/PINN/Analytical fallback),
- short interpretation.

6.2 Scenario B: Tidally locked habitable candidate

Run a Proxima Cen b-like configuration (or choose from Catalog examples), then evaluate:

- day-night temperature structure,
- habitable surface fraction,
- false-positive risk indicators,
- climate-state interpretation.

6.3 Scenario C: Extreme/hot rocky control

Use high instellation and short orbit case to demonstrate non-habitable behavior and verify that metrics react consistently.

6.4 Scenario D: Catalog-scale anomaly analysis

In Catalog tab, fetch full candidates, run anomaly detection, and report:

- number of analyzed rows,
- number/rate of anomalies,
- short interpretation of "weirdest planets" table.

Suggested results table (fill with your run outputs)

Scenario	Key input profile	(T_{eq}) [K]	ESI	SEPHI	HSF	Main interpretation
A	Earth-like	[fill]	[fill]	[fill]	[fill]	[fill]
B	Tidally locked candidate	[fill]	[fill]	[fill]	[fill]	[fill]
C	Hot rocky control	[fill]	[fill]	[fill]	[fill]	[fill]
D	Catalog anomalies	[fill]	[n/a]	[n/a]	[n/a]	[fill]

Also include a brief diagnostics summary from the System tab (NASA API check, Earth (T_{eq}) sanity, validator check, model availability).

Figure 11. [Insert screenshot: Earth-like simulation outputs.]

Figure 12. [Insert screenshot: Tidally locked scenario outputs.]

Figure 13. [Insert screenshot: Anomaly detection and UMAP results.]

7. Conclusions

The Hack-4-Sages project demonstrates a practical hybrid architecture for exoplanet analysis:

- deterministic astrophysics for scientific grounding,
- surrogate modeling for speed,
- optional LLM layers for explanation and interaction,
- and robust degradation/validation for reliability.

The system is strongest as a rapid exploration and decision-support platform: it enables quick comparative analysis, visual interpretation, and citation-supported narrative generation directly in one interface.

At the same time, the project explicitly acknowledges important limitations (surrogate nature, simplified assumptions compared with full GCM pipelines, and dependency on available model/data quality). This makes the output scientifically honest and suitable for educational, exploratory, and hackathon demonstration contexts.

Recommended future improvements include deeper calibration against external climate references, broader uncertainty benchmarking, and tighter integration of observational updates.

Literature

1. Kasting, J.F., Whitmire, D.P., Reynolds, R.T. (1993). *Habitable Zones Around Main Sequence Stars*. Icarus, 101, 108-128.
2. Schulze-Makuch, D. et al. (2011). *A Two-Tiered Approach to Assessing the Habitability of Exoplanets*. Astrobiology, 11(10), 1041-1052.
3. Kopparapu, R.K. et al. (2013). *Habitable Zones Around Main-Sequence Stars: New Estimates*. ApJ, 765, 131.
4. Rodriguez-Mozos, J.M., Moya, A. (2017). *SEPHI*. MNRAS, 471(4), 4628-4636.
5. Huang, G.-B. et al. (2006). *Extreme Learning Machine*. Neurocomputing, 70, 489-501.
6. Liu, F.T. et al. (2008). *Isolation Forest*. ICDM 2008.

7. Kite, E.S. et al. (2009). *Geophysical Controls on Volcanism/Outgassing*. ApJ, 700, 1732.
8. Leconte, J. et al. (2013). *3D Climate Modeling of Close-in Land Planets*. A&A, 554, A69.
9. Yang, J. et al. (2013). *Cloud Feedback and Habitable Zone Expansion*. ApJ Letters, 771, L45.
10. Hu, Y., Yang, J. (2014). *Ocean Heat Transport on Tidally Locked Planets*. J. Climate, 27(19), 7684-7697.
11. Luger, R., Barnes, R. (2015). *Extreme Water Loss and Abiotic O₂*. Astrobiology, 15(2), 119-143.
12. Meadows, V.S. et al. (2018). *Oxygen as a Biosignature in Context*. Astrobiology, 18(6), 630-662.
13. Chen, J., Kipping, D.M. (2017). *Probabilistic Forecasting of the Masses and Radii of Exoplanets*. ApJ, 834, 17.
14. McInnes, L. et al. (2018). *UMAP: Uniform Manifold Approximation and Projection*. JOSS, 3(29), 861.
15. Relevant project documentation: [README.md](#), [METHODOLOGY.md](#), and implementation modules in [modules/](#).